[Nome dell'Università]

[Dipartimento]

[Corso di Laurea in Ingegneria Elettronica / Informatica]

Interfacciamento di un coprocessore video VERA al bus PBI dell'Atari XL/XE tramite ESP32-S3

Architettura del firmware, emulazione della RAMbo 320K
e verifica temporale a 1,77 e 1,79 MHz

Relatore:
[Prof. Nome Cognome]

Correlatore:
[Nome Cognome]

Candidato:
Gianluca Renzi
[Matricola]

Anno Accademico 2025/2026

Sommario

Questa tesi descrive e verifica il firmware del modulo *Vera-Module-RBL-XE*, una scheda di espansione per i computer Atari 8-bit della serie XL/XE che collega al bus del 6502 il coprocessore video VERA del progetto Commander X16. Un microcontrollore ESP32-S3 osserva ogni ciclo del bus, decodifica in software tutti i sedici bit di indirizzo e, a seconda dell'indirizzo, abilita i registri della VERA, fornisce la ROM e la RAM del dispositivo PBI (Parallel Bus Interface) ed emula un'espansione di memoria RAMbo da 256 KB (configurazione «RAMbo 320K»: 64 KB di base più 16 banchi da 16 KB).

Il lavoro ha due obiettivi. Il primo è documentare l'architettura hardware e software del modulo, il protocollo PBI e il meccanismo di bank switching della RAMbo. Il secondo è verificare che il firmware rispetti le tempistiche del bus a 1,773 447 MHz (PAL) e 1,789 773 MHz (NTSC). Per la verifica si costruisce un modello temporale del percorso dal fronte di ϕ_2 al dato sul bus. Il modello parte dal conteggio delle istruzioni nel disassemblato del firmware compilato e dai parametri dei datasheet del 6502, della logica di traslazione dei livelli e della FPGA.

L'analisi mostra che la decodifica e l'asserzione dei segnali di controllo sono realizzabili, mentre i cicli di lettura serviti direttamente dall'ESP32 (ROM PBI `$D800`–`$DFFF`, RAM PBI `$D600`–`$D7FF` e finestra RAMbo `$4000`–`$7FFF`) non rispettano il tempo di setup del 6502 con la struttura attuale del ciclo principale: anche con la latenza di accesso alle periferiche più favorevole il dato arriva sul bus circa 331 ns dopo il fronte di salita di ϕ_2, contro un limite di 229 ns. L'analisi ha inoltre messo in luce due difetti indipendenti dalle tempistiche: l'array della RAMbo e la ROM PBI sono collocati in un'area di memoria non accessibile a byte, e l'array della RAMbo si sovrappone fisicamente ai dati e allo heap del firmware. Per ogni problema la tesi propone una correzione e ne stima l'effetto; chiude un piano di verifica sperimentale con analizzatore logico.

Indice

Sommario **i**

1 Introduzione **1**
1.1 Contesto . . . 1
1.2 Obiettivi . . . 1
1.3 Metodo . . . 2
1.4 Struttura della tesi . . . 2

2 Il bus dell'Atari XL/XE **3**
2.1 Il processore e il clock di sistema . . . 3
2.2 Il ciclo di bus del 6502 . . . 3
2.2.1 Parametri temporali . . . 4
2.3 Il connettore PBI/ECI e la gestione della memoria . . . 4

3 Architettura hardware del modulo **7**
3.1 Schema a blocchi . . . 7
3.2 Assegnazione dei pin dell'ESP32-S3 . . . 8
3.3 Logica di traslazione e ritardi . . . 8

4 Protocollo PBI ed espansione RAMbo 320K **11**
4.1 Il dispositivo PBI . . . 11
4.1.1 Selezione tramite `$D1FF` . . . 11
4.1.2 La ROM di gestione . . . 11
4.1.3 Mappa degli indirizzi . . . 11
4.2 L'espansione RAMbo 320K . . . 12
4.2.1 Bank switching tramite PORTB . . . 12
4.2.2 Osservazione passiva della PIA . . . 12

5 Architettura del firmware **15**
5.1 Divisione del lavoro fra i due core . . . 15
5.2 Sequenza di avvio . . . 15
5.3 Decodifica a tabelle . . . 15
5.4 Il ciclo principale . . . 16
5.5 Analisi del codice generato . . . 17

6 Verifica temporale **19**
6.1 Modello . . . 19
6.1.1 La latenza degli accessi alle periferiche . . . 19
6.2 Rilevamento del fronte di ϕ_2 . . . 19
6.3 Ciclo di lettura servito dall'ESP32 . . . 20

6.4 Ciclo di lettura servito dalla VERA . . . 21
6.5 Cicli di scrittura . . . 21
6.5.1 Campionamento del dato . . . 21
6.5.2 Effetti della scrittura durante ϕ_1 . . . 21
6.5.3 Hold del dato verso la VERA . . . 22
6.6 Asserzione di EXTSEL e MPD . . . 22
6.7 Interruzioni sul core 0 . . . 23
6.8 Sintesi grafica . . . 23

7 Difetti non temporali **25**
7.1 Accessi a byte all'area IRAM . . . 25
7.2 Sovrapposizione fra RAMbo e memoria dati . . . 25
7.3 ROM di autotest e finestra RAMbo . . . 26
7.4 Altri punti minori . . . 26

8 Proposte di correzione **27**
8.1 Correzioni della collocazione in memoria . . . 27
8.2 Ciclo principale con decodifica durante ϕ_1 . . . 27
8.3 Isolamento del ciclo principale dalle interruzioni . . . 28
8.4 Correzioni minori del firmware . . . 29
8.5 Correzioni hardware . . . 29
8.6 Alternativa architetturale: ROM, RAM e RAMbo nella FPGA di interfaccia 29

9 Piano di verifica sperimentale **31**
9.1 Misura della latenza APB . . . 31
9.2 Misure sul bus con analizzatore logico . . . 31
9.3 Prove funzionali . . . 32

10 Conclusioni **33**

A Il modello temporale **37**

Elenco delle figure

2.1 Ciclo di bus del 6502 a 1,79 MHz 5

3.1 Schema a blocchi del modulo 7

5.1 Flusso del ciclo principale di `MonitorTask` per un ciclo di bus. 16

6.1 Lettura servita dall'ESP32 (firmware attuale) 20
6.2 Hold del dato di scrittura verso la VERA 22
6.3 Effetto dell'interruzione del tick di FreeRTOS sul core 0. 23
6.4 Margine dei percorsi critici in funzione della latenza APB (NTSC). 24
6.5 Margine dei percorsi critici in funzione della latenza APB (PAL). 24

8.1 Ciclo principale proposto 28

Elenco delle tabelle

2.1 Clock del 6502 negli Atari XL/XE e finestra di ϕ_2 alto. 3
2.2 Parametri temporali del 6502 (datasheet Rockwell R6502). 4

3.1 Assegnazione dei segnali del bus ai GPIO dell'ESP32-S3. 8
3.2 Ritardi della logica di interfaccia usati nel modello. 9

4.1 Indirizzi gestiti dal firmware in modalità PBI. 12
4.2 Valori di PORTB per i 16 banchi (bit 0, 1 e 7 a 1, bit 4 a 0). 12
4.3 Inizializzazione della PIA da parte del sistema operativo e reazione del firmware. 13

5.1 Cicli di CPU lungo i percorsi del ciclo principale (dal disassemblato). 18

6.1 Lettura servita dall'ESP32: istante del dato sul bus e margine. 20
6.2 Lettura di un registro VERA: istante del dato sul bus e margine. 21
6.3 Esito della verifica temporale. 23

8.1 Ciclo proposto, caso peggiore: istanti rispetto a $\phi_2 \uparrow$ e margine del dato. . . 28

Capitolo 1

Introduzione

1.1 Contesto

I computer Atari 400/800 e le successive serie XL e XE, prodotti fra il 1979 e il 1992, sono basati sul microprocessore MOS 6502 (nella versione «SALLY» 6502C) e su tre circuiti dedicati: ANTIC per la generazione del display tramite DMA, GTIA per il colore e gli sprite, POKEY per l'audio, la tastiera e la comunicazione seriale. Con la serie XL Atari introdusse il connettore *Parallel Bus Interface* (PBI), sostituito nella serie XE dall'*Enhanced Cartridge Interface* (ECI), che porta all'esterno il bus del processore insieme a un piccolo insieme di segnali di controllo: con questi un dispositivo esterno può sostituire porzioni della memoria interna e aggiungere una propria ROM di gestione che il sistema operativo riconosce all'avvio [1].

La VERA (*Versatile Embedded Retro Adapter*) è il coprocessore video e audio del progetto Commander X16, realizzato su una FPGA Lattice iCE40UP5K [11]. Offre 128 KB di VRAM, due layer grafici, 128 sprite, uscita VGA/HDMI e un generatore audio, ed espone al processore 32 registri a 8 bit. Il modulo *Vera-Module-RBL-XE*, oggetto di questa tesi, porta la VERA su un Atari XL/XE attraverso il connettore PBI/ECI.

1.2 Obiettivi

Il lavoro si concentra sul firmware del microcontrollore ESP32-S3 che controlla il bus. Gli obiettivi sono:

1. descrivere l'architettura hardware e software del modulo e il ruolo di ogni segnale del bus;

2. descrivere i due servizi che il firmware offre all'Atari oltre all'accesso alla VERA: il dispositivo PBI (ROM e RAM di gestione, selezione tramite `$D1FF`) e l'emulazione dell'espansione RAMbo 320K;

3. verificare che il firmware rispetti le tempistiche del bus del 6502 alle due frequenze di clock dei sistemi PAL (1,77 MHz) e NTSC (1,79 MHz), individuando i percorsi critici;

4. rappresentare con diagrammi temporali i segnali critici;

5. proporre le correzioni necessarie e un piano di verifica sperimentale.

1.3 Metodo

La scheda non è ancora stata provata su un Atari reale: la documentazione del progetto lo dichiara esplicitamente [14]. La verifica è quindi *analitica*. Per il lato ESP32 il punto di partenza è il codice macchina effettivamente generato: il firmware è stato compilato con la stessa configurazione di produzione (PlatformIO, piattaforma `espressif32@6.6.0`, arduino-esp32 2.0.14 su ESP-IDF 4.4.6, ottimizzazione `-O3`) e la funzione del ciclo principale è stata disassemblata per contare le istruzioni lungo ogni percorso. Per il lato Atari e per la logica di interfaccia si sono usati i parametri dei datasheet. Le grandezze che i documenti disponibili non specificano, in particolare la latenza di accesso della CPU ai registri GPIO dell'ESP32-S3, sono trattate come parametri: i risultati sono presentati in funzione di quel parametro, così da distinguere le conclusioni che valgono in ogni caso da quelle che dipendono da una misura ancora da fare.

Tutti i numeri delle tabelle e dei grafici sono prodotti da un unico script, `timing_model.py`, allegato alla tesi: cambiare un parametro e ricompilare il documento aggiorna tabelle, testo e figure in modo coerente.

1.4 Struttura della tesi

Il capitolo 2 descrive il bus dell'Atari XL/XE, il ciclo del 6502 e i parametri temporali di riferimento. Il capitolo 3 presenta l'hardware del modulo. Il capitolo 4 descrive il protocollo PBI e l'espansione RAMbo 320K. Il capitolo 5 descrive il firmware e ne analizza il codice generato. Il capitolo 6 contiene la verifica temporale vera e propria; il capitolo 7 riporta i difetti non temporali emersi durante l'analisi. Il capitolo 8 propone le correzioni, il capitolo 9 il piano di verifica sperimentale e il capitolo 10 le conclusioni.

Capitolo 2

Il bus dell'Atari XL/XE

2.1 Il processore e il clock di sistema

Gli Atari XL/XE usano il 6502C «SALLY», una versione del 6502 NMOS con la logica di HALT integrata: ANTIC usa HALT per sospendere il processore durante i propri cicli di DMA [1]. Il clock del processore deriva dall'oscillatore del colore: nei sistemi NTSC il 6502 gira a metà della frequenza della sottoportante colore, nei sistemi PAL ai due quinti della sottoportante [1, §2.2]. Ne risultano le grandezze della tabella 2.1.

Tabella 2.1: Clock del 6502 negli Atari XL/XE e finestra di ϕ_2 alto.

	PAL	**NTSC**
Frequenza $f_{\phi 2}$	1,773 447 MHz	1,789 773 MHz
Periodo $T = 1/f_{\phi 2}$	563,9 ns	558,7 ns
ϕ_2 alto, $T_H = T/2$	281,9 ns	279,4 ns
Cicli per riga di scansione	114	114
Cicli per quadro	35 568	29 868
Cicli ESP32-S3 a 240 MHz in T_H	67,7	67

La prima bozza di questo lavoro usava 559 ns per entrambi i sistemi: il periodo PAL è in realtà più lungo di circa 5 ns e la finestra di ϕ_2 alto di circa 2,6 ns. La differenza è piccola rispetto agli altri termini del bilancio temporale (capitolo 6): il sistema NTSC, più veloce, è il caso peggiore e quello di riferimento.

Il clock ϕ_0 generato da ANTIC si ottiene dividendo per due il clock del colore ed è quindi nominalmente simmetrico; il ϕ_2 in uscita dal processore, che è quello presente sul connettore PBI, può discostarsi dal 50 % di qualche decina di nanosecondi. Nell'analisi del caso peggiore si assume una riduzione di 10 ns della fase alta (parametro `PHI2_ASYM` del modello), da sostituire con il valore misurato.

2.2 Il ciclo di bus del 6502

Il 6502 esegue un accesso alla memoria per ciclo di clock, senza stati di attesa. Il ciclo si divide in due fasi:

ϕ_1 **(ϕ_2 basso)** il processore porta sul bus il nuovo indirizzo e il livello di $R/\overline{W}$; il bus dati non è usato;

ϕ_2 **alto** la memoria o la periferica indirizzata risponde. In lettura il processore campiona il bus dati sul fronte di discesa di ϕ_2; in scrittura il processore pilota il bus dati, che i dispositivi campionano entro la fine della fase.

Il 6502 non ha una linea READY utilizzabile dai dispositivi lenti negli Atari (RDY è riservata ad ANTIC): un dispositivo che non risponde in tempo produce semplicemente un dato errato. Per lo stesso motivo nessun ciclo può essere allungato.

2.2.1 Parametri temporali

Il datasheet di SALLY non è pubblico. Poiché il processore funziona a 1,79 MHz, i suoi parametri devono essere almeno quelli della versione a 2 MHz del 6502; la tabella 2.2 riporta i valori del datasheet Rockwell R6502 [2], che sono quelli usati in tutta l'analisi. Per confronto è indicata anche la versione a 1 MHz.

Tabella 2.2: Parametri temporali del 6502 (datasheet Rockwell R6502).

Simbolo	Parametro	1 MHz	2 MHz	
t_{ADS}	indirizzo valido dopo l'inizio di ϕ_1	225	140	max
t_{RWS}	$R/\overline{W}$ valido dopo l'inizio di ϕ_1	225	140	max
t_{ACC}	tempo di accesso disponibile alla memoria	650	310	min
t_{DSU}	setup del dato in lettura prima di $\phi_2 \downarrow$	100	50	min
t_{HR}	hold del dato in lettura dopo $\phi_2 \downarrow$	10	10	min
t_{MDS}	dato di scrittura valido dopo $\phi_2 \uparrow$	175	100	max
t_{HW}	hold del dato di scrittura dopo $\phi_2 \downarrow$	30	30	min
t_{HA}	hold dell'indirizzo dopo $\phi_2 \downarrow$	30	30	min

Valori in nanosecondi.

Da questi valori derivano i due limiti che dominano l'analisi:

- in un ciclo di lettura il dato deve essere stabile sul bus entro

$$t_{\text{dato}} \leq T_H - t_{DSU} \tag{2.1}$$

  dal fronte di salita di ϕ_2: 229 ns in NTSC e 232 ns in PAL con ϕ_2 simmetrico, 219 ns e 222 ns con la riduzione di 10 ns della fase alta;

- in un ciclo di scrittura il dato del processore è garantito solo fra $t_{MDS} = 100$ ns dopo la salita di ϕ_2 e $t_{HW} = 30$ ns dopo la discesa.

L'indirizzo, invece, è stabile già durante ϕ_1: al più 140 ns dopo l'inizio di ϕ_1, cioè circa 140 ns *prima* della salita di ϕ_2. Questo margine non è sfruttato dal firmware attuale e sarà la base della principale proposta del capitolo 8.

La figura 2.1 riporta in scala un ciclo di lettura e uno di scrittura a 1,79 MHz.

2.3 Il connettore PBI/ECI e la gestione della memoria

Il connettore PBI degli XL porta il bus indirizzi e dati, ϕ_2, $R/\overline{W}$, IRQ, RESET, HALT, REFRESH e tre segnali che un dispositivo esterno usa per sovrapporsi alla memoria interna [1, cap. 10]:

EXTSEL (attivo basso) disabilita la RAM interna nel ciclo corrente: il dispositivo esterno risponde al suo posto;

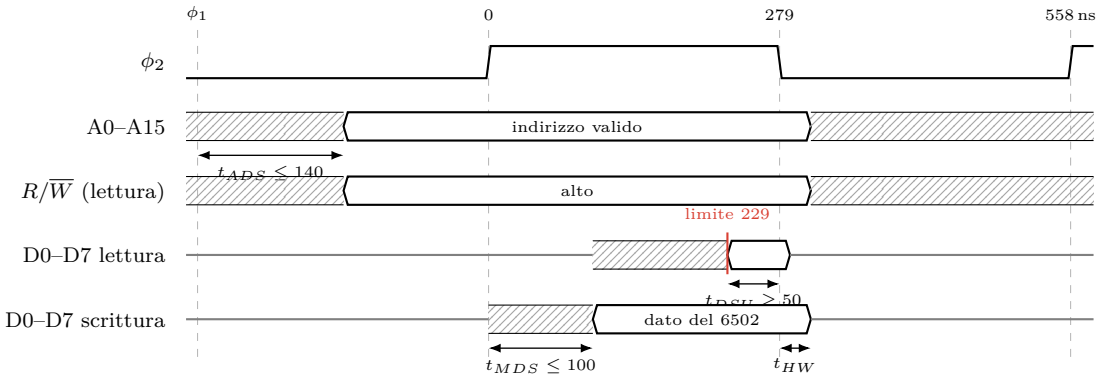


Figura 2.1: Ciclo di bus del 6502 a 1,79 MHz (NTSC) in scala, con i parametri della versione a 2 MHz. Le zone tratteggiate sono valori non garantiti; la riga grigia indica il bus in alta impedenza. L'indirizzo è valido circa 140 ns prima della salita di ϕ_2; in lettura il dato deve essere stabile da 229 ns.

MPD (*Math Pack Disable*, attivo basso) disabilita la ROM interna del Math Pack a `$D800`–`$DFFF`: è la finestra in cui un dispositivo PBI mappa la propria ROM di gestione;

D1XX decodifica della pagina `$D1xx` riservata ai dispositivi PBI.

La logica di mappatura della memoria (MMU) decide il dispositivo interno da abilitare solo in base all'indirizzo e allo stato di PORTB [1, §3.4]. Un dispositivo esterno può sovrapporsi alla RAM e, tramite MPD, alla sola ROM del Math Pack, ma non all'I/O né allo spazio della cartuccia.

I dispositivi PBI commerciali (ICD Multi I/O, CSS Black Box, Atari 1090) decodificano l'indirizzo con logica combinatoria: EXTSEL e MPD sono quindi validi poche decine di nanosecondi dopo che l'indirizzo è stabile, cioè ancora durante ϕ_1. La documentazione disponibile non fissa un limite temporale esplicito per questi segnali rispetto alla generazione dei segnali RAS/CAS della RAM dinamica e alla selezione delle ROM interne; il punto è discusso nella sezione 6.6.

Capitolo 3

Architettura hardware del modulo

3.1 Schema a blocchi

La figura 3.1 riassume l'hardware descritto in [13, 14]. Il bus Atari è a 5 V TTL, mentre ESP32-S3 e FPGA sono a 3,3 V: tutti i segnali attraversano traslatori di livello 74LVC4245 (otto canali, direzione impostata da un pin DIR e abilitazione $\overline{\text{OE}}$).

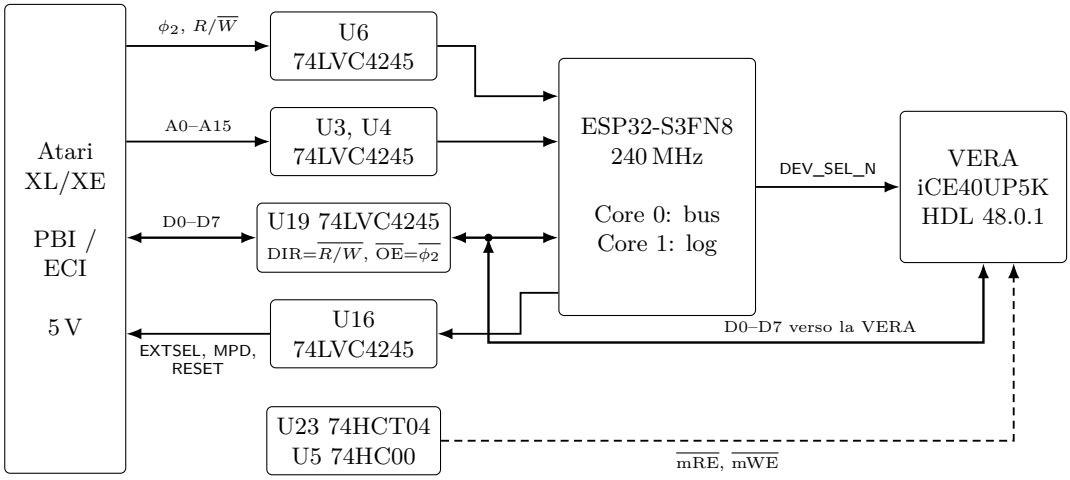


Figura 3.1: Schema a blocchi semplificato del modulo Vera-Module-RBL-XE (riferimenti dei componenti dalla netlist del 2026-09-01).

U6 traslatore Atari→scheda per ϕ_2, $R/\overline{W}$, D1XX, S4, S5, CCTL, REFRESH;

U3, U4 traslatori del bus indirizzi (A8–A15 e A0–A7);

U19 traslatore bidirezionale del bus dati. DIR è $\overline{R/W}$ e $\overline{\text{OE}}$ è $\overline{\phi_2}$, entrambi generati dagli invertitori 74HCT04 di U23: U19 è abilitato in ogni ciclo, per tutta la fase alta di ϕ_2;

U16 traslatore scheda→Atari per EXTSEL, MPD, RESET, HALT, RD4, RD5, IRQ;

U5 porte NAND 74HC00 che generano gli strobe della VERA: $\overline{\text{mRE}} = \overline{\phi_2 \cdot R/W}$ e $\overline{\text{mWE}} = \overline{\phi_2 \cdot \overline{R/W}}$;

ESP32-S3FN8 microcontrollore dual core Xtensa LX7 a 240 MHz con 8 MB di flash integrata e 512 KB di SRAM [4]; il chip select della VERA, DEV_SEL_N, va direttamente alla FPGA a 3,3 V;

VERA FPGA iCE40UP5K con il firmware VERA 48.0.1; la configurazione è caricata da una flash SPI all'uscita dal reset (CRESET) e il pin CDONE ne segnala il completamento.

3.2 Assegnazione dei pin dell'ESP32-S3

L'ESP32-S3FN8 nel package QFN56 offre 45 GPIO, ma sei sono occupati dalla flash interna e quattro sono pin di strapping letti al reset. Il bus del 6502 richiede 26 segnali di ingresso (16 indirizzi, 8 dati, ϕ_2, $R/\overline{W}$) e 3 uscite di controllo: la tabella 3.1 riporta l'assegnazione [13]. Restano liberi solo GPIO 47 e 48, troppo pochi per usare i segnali di decodifica hardware presenti sul connettore (D1XX, S4, S5, CCTL): per questo la decodifica dell'indirizzo è interamente software.

Tabella 3.1: Assegnazione dei segnali del bus ai GPIO dell'ESP32-S3.

Segnale	GPIO	Registro	Note
ϕ_2	1	`GPIO.in` bit 1	e Dedicated GPIO, ingresso 0
$R/\overline{W}$	2	`GPIO.in` bit 2	
RAMBO_EN	3	`GPIO.in` bit 3	strapping, letto all'avvio
D0–D7	4–11	`GPIO.in` 4–11	bidirezionale
A0–A7	12–19	`GPIO.in` 12–19	GPIO 19 = USB D−
A8, A9	20, 21	`GPIO.in` 20–21	GPIO 20 = USB D+
A12, A13, A10, A11	33–36	`GPIO.in1` 1–4	
A14, A15	45, 46	`GPIO.in1` 13–14	strapping
ARESET, CRESET	37, 38	uscite	reset di Atari e VERA
CDONE	39	`GPIO.in1` bit 7	configurazione VERA completata
DEV_SEL_N	40	Dedicated GPIO uscita 0	chip select VERA
EXTSEL_N	41	Dedicated GPIO uscita 1	verso il bus via U16
MPD	42	Dedicated GPIO uscita 2	verso il bus via U16
UART0 TX, RX	43, 44		log di debug

Il campionamento dello stato del bus richiede quindi due letture di registri di periferica: `GPIO.in` (GPIO 0–31: ϕ_2, $R/\overline{W}$, dati, A0–A9) e `GPIO.in1` (GPIO 32–48: A10–A15). I tre segnali di controllo sono invece collegati al modulo *Dedicated GPIO* dell'ESP32-S3, che la CPU legge e scrive con istruzioni dedicate (`ee.get_gpio_in`, `ee.wr_mask_gpio_out`) in un ciclo di clock, senza passare dal bus delle periferiche [5, 6].

3.3 Logica di traslazione e ritardi

La tabella 3.2 riporta i ritardi usati nel modello. I valori minimo e massimo vengono dai datasheet dei componenti nelle condizioni di alimentazione della scheda; per il 74HC00 alimentato a 3,3 V il datasheet fornisce solo i valori a 2 e 4,5 V e quelli in tabella sono interpolati. Il valore per la FPGA è una stima del percorso pad–logica–pad dell'iCE40UP5K per un'uscita combinatoria.

Tabella 3.2: Ritardi della logica di interfaccia usati nel modello.

Componente	**Percorso**	**min**	**tip**	**max**
74LVC4245A [7]	propagazione 5 $\leftrightarrow$ 3,3 V	1,0	4,0	6,5
74LVC4245A	disabilitazione uscita	1,0	4,0	8,0
74HCT04 [8]	propagazione a 5 V	4	10	24
74HC00 [9]	propagazione a 3,3 V (interp.)	5	12	30
iCE40UP5K [10]	pad $\rightarrow$ logica $\rightarrow$ pad (stima)	6	10	15
ESP32-S3	sincronizzatore di ingresso GPIO	0	12,5	25
ESP32-S3	uscita GPIO verso il pad	2	4	6

Valori in nanosecondi.

Capitolo 4

Protocollo PBI ed espansione RAMbo 320K

4.1 Il dispositivo PBI

4.1.1 Selezione tramite `$D1FF`

Più dispositivi PBI possono essere collegati in catena. Il registro `$D1FF` (PDVS in scrittura) seleziona il dispositivo attivo: ogni dispositivo è associato a un bit e scrivere un valore con il proprio bit a 1 lo seleziona, scrivere `$00` deseleziona tutti [1, §10.2]. Il registro non esiste nel computer: ogni dispositivo lo implementa catturando le scritture all'indirizzo. Il modulo usa il bit 7 (`PBI_DEV_ID = 0x80`) e, come richiede la regola PBI, considera solo il proprio bit: `selected = (data & 0x80) != 0`.

Quando un dispositivo è selezionato:

- la sua ROM appare a `$D800`–`$DFFF` al posto del Math Pack (MPD);
- la pagina `$D1xx` e l'area `$D600`–`$D7FF` appartengono a lui (EXTSEL);

quando non è selezionato non deve rispondere a nessuno di questi indirizzi, così da poter convivere con altri dispositivi PBI. Nel modulo i registri della VERA occupano `$D100`–`$D11F` e sono quindi accessibili solo con la scheda selezionata (tabella 4.1).

4.1.2 La ROM di gestione

All'avvio il sistema operativo XL/XE seleziona a turno ogni bit di `$D1FF`, verifica l'intestazione della ROM e, se valida, chiama la routine di inizializzazione del dispositivo. La ROM del modulo (`vera_pbi_handler.s`, 2 KB assemblati con `ca65` all'indirizzo `$D800`) espone l'intestazione richiesta: `$D803` = `$80` (il bit del dispositivo), `$D80B` = `$91` (firma), `$D80C` = `'V'`, e i vettori `$D805` (SIO, che restituisce «non gestito») e `$D808` (IRQ). Ogni lettura di questi byte è un ciclo di lettura servito dall'ESP32: la correttezza temporale di questi cicli (sezione 6.3) è quindi la condizione perché la scheda venga riconosciuta.

4.1.3 Mappa degli indirizzi

In modalità CCTL (cartuccia, `VERA_BOARD_IS_PBI = 0`) il latch è a `$D5FF` e i registri della VERA occupano `$D500`–`$D5FE`; non ci sono ROM e RAM PBI.

Tabella 4.1: Indirizzi gestiti dal firmware in modalità PBI.

Intervallo	Condizione	Segnali	Dato
`$D100`–`$D11F`	scheda selezionata	EXTSEL, DEV_SEL_N	VERA
`$D120`–`$D1FE`	scheda selezionata	EXTSEL	non pilotato
`$D1FF`	scrittura	—	latch di selezione
`$D301`, `$D303`	scrittura	—	osservazione PIA
`$D600`–`$D7FF`	scheda selezionata	EXTSEL	ESP32, RAM 512 B
`$D800`–`$DFFF`	scheda selezionata	MPD	ESP32, ROM 2 KB
`$4000`–`$7FFF`	RAMbo attiva	EXTSEL	ESP32, banco 16 KB

4.2 L'espansione RAMbo 320K

4.2.1 Bank switching tramite PORTB

Negli XL/XE la porta B della PIA 6520 (`$D301`, PORTB) non serve più ai joystick ma controlla la mappa della memoria: il bit 0 abilita la ROM del sistema operativo, il bit 1 il BASIC, il bit 7 la ROM di autotest a `$5000`–`$57FF` [1, §3.4]. Il 130XE usa i bit 2–5 per un'espansione da 64 KB vista attraverso la finestra `$4000`–`$7FFF`.

L'espansione «320K Rambo» usa i bit 2, 3, 5 e 6 di PORTB per selezionare uno di 16 banchi da 16 KB e il bit 4 per abilitare la finestra: $64 + 16 \times 16 = 320$ KB [1, §3.5]. Poiché il bit 5 serve per il numero di banco, CPU e ANTIC vedono sempre lo stesso banco: non esiste l'accesso separato di ANTIC del 130XE. Il firmware calcola il banco come

$$\text{banco} = ((\text{PORTB} \gg 2)\ \&\ 3)\ |\ ((\text{PORTB} \gg 3)\ \&\ 0\text{x}0\text{C}) \tag{4.1}$$

che corrisponde alla configurazione «320K Rambo» documentata per Altirra («PORTB bits 2, 3, 5, and 6 control bank; bit 4 controls CPU+ANTIC access» [1, §11]). Poiché l'ESP32 non distingue i cicli della CPU da quelli di DMA di ANTIC (non osserva HALT), risponde a entrambi con lo stesso banco: è esattamente il comportamento della RAMbo 320K.

Tabella 4.2: Valori di PORTB per i 16 banchi (bit 0, 1 e 7 a 1, bit 4 a 0).

Banco	PORTB	Banco	PORTB	Banco	PORTB	Banco	PORTB
0	`$83`	4	`$A3`	8	`$C3`	12	`$E3`
1	`$87`	5	`$A7`	9	`$C7`	13	`$E7`
2	`$8B`	6	`$AB`	10	`$CB`	14	`$EB`
3	`$8F`	7	`$AF`	11	`$CF`	15	`$EF`

4.2.2 Osservazione passiva della PIA

La PIA fisica resta sulla scheda madre e risponde normalmente a `$D301` e `$D303`: l'ESP32 non la emula ma ne *osserva* le scritture. A `$D301` la PIA espone due registri diversi, il registro di uscita e il registro di direzione (DDR), selezionati dal bit 2 del registro di controllo PBCTL (`$D303`). Sul bus le due scritture sono indistinguibili; il firmware tiene quindi una copia di PBCTL e aggiorna la propria copia di PORTB solo quando il bit 2 di PBCTL è a 1. La tabella 4.3 mostra la sequenza tipica del sistema operativo.

Tabella 4.3: Inizializzazione della PIA da parte del sistema operativo e reazione del firmware.

Passo	Scrittura	PIA	Firmware
1	`$00` → `$D303`	DDR selezionato a `$D301`	`pbctl = 0x00`
2	`$FF` → `$D301`	tutti i pin in uscita	ignorata
3	`$34` → `$D303`	registro di uscita a `$D301`	`pbctl = 0x34`
4	`$NN` → `$D301`	nuova mappa di memoria	`portb = 0xNN`, tabella aggiornata

Capitolo 5

Architettura del firmware

5.1 Divisione del lavoro fra i due core

Il firmware è un'applicazione Arduino su ESP-IDF e FreeRTOS. Il core 0 esegue solo il task `MonitorTask`, con la priorità massima, che non cede mai la CPU; il core 1 esegue `setup()` e `loop()`, quest'ultimo dedicato a stampare sulla seriale gli eventi prodotti dal core 0. Gli eventi passano attraverso un buffer circolare senza lock (un produttore, un consumatore): l'inserimento costa poche istruzioni, mentre `xQueueSend()` richiedeva circa 1 μs e faceva perdere i cicli di bus successivi.

5.2 Sequenza di avvio

1. `setup()` porta subito a livello basso ARESET e CRESET: Atari e VERA restano in reset per tutta l'inizializzazione;
2. legge RAMBO_EN (GPIO 3) per sapere se l'emulazione RAMbo è abilitata e costruisce le tabelle di decodifica;
3. `MonitorTask` rilascia CRESET, attende CDONE (timeout 5 s), attende ancora 1 ms per il reset interno della VERA e rilascia ARESET: da questo momento il bus è attivo e il ciclo principale deve rispondere a ogni ciclo di ϕ_2.

5.3 Decodifica a tabelle

Confrontare l'indirizzo con gli intervalli della tabella 4.1 a ogni ciclo costava circa 95 istruzioni. La versione attuale precalcola tutto ciò che dipende dalla pagina (A8–A15) e dallo stato della scheda (selezionata o no, RAMbo attiva o no) in quattro tabelle da 256 voci (`bus_decode.h`). L'indice di pagina si ottiene dai due campioni dei registri GPIO senza riordinare i bit:

```
static inline uint32_t bus_page_idx(uint32_t lo, uint32_t hi)
{
    return ((lo >> 20) & 0x03u) | ((hi << 1) & 0x3Cu) | ((hi >> 7) & 0xC0u);
}
```

Listing 5.1: Indice di pagina dai registri GPIO (`bus_decode.h`).

Ogni voce contiene nei bit 0–2 i livelli di DEV_SEL_N, EXTSEL_N e MPD, pronti per `ee.wr_mask_gpio_out`, e nei bit 3–7 le azioni: `CT_SLOW` (la decisione dipende anche da A0–A7,

solo per `$D1xx`/`$D5xx`), `CT_DRIVE` (pilotare il bus in lettura), `CT_RAMBO` (aggiungere l'offset del banco), `CT_WR_MEM` (memorizzare il dato in scrittura), `CT_WR_REG` (latch o osservazione della PIA). Il passaggio fra gli stati cambia solo il puntatore alla tabella corrente. La logica non dipende da ESP-IDF ed è verificata sul PC contro la versione a confronti di intervalli per tutti i 65 536 indirizzi e i quattro stati.

5.4 Il ciclo principale

La figura 5.1 mostra il flusso di un ciclo di bus e il listato 5.2 il codice sorgente.

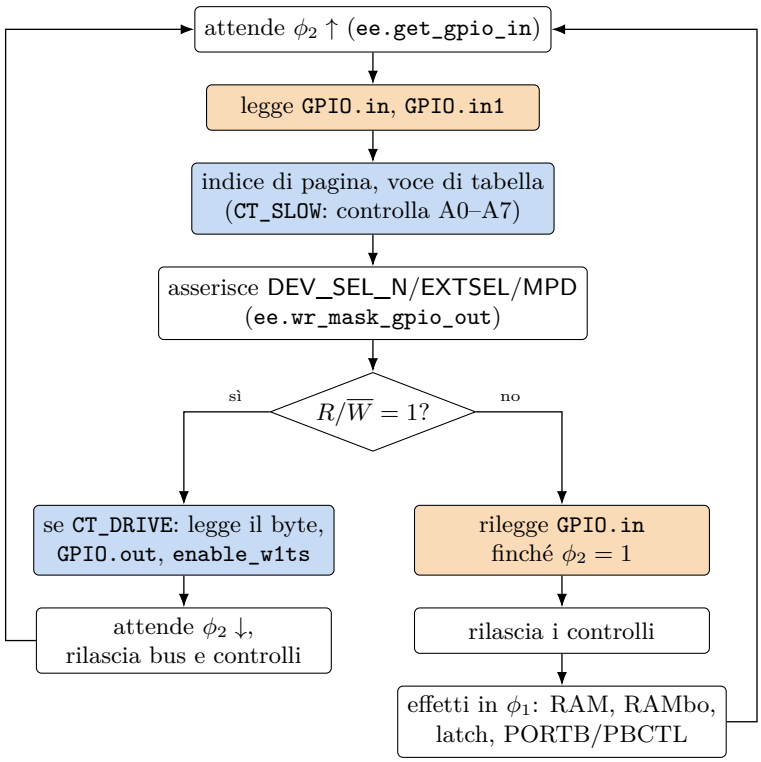


Figura 5.1: Flusso del ciclo principale di `MonitorTask` per un ciclo di bus.

```
for (;;)
{
    while (!(cpu_ll_read_dedic_gpio_in() & DEDIC_IN_PHI2));
    uint32_t g_lo = GPIO.in;
    uint32_t g_hi = GPIO.in1.val;

    uint32_t idx  = bus_page_idx(g_lo, g_hi);
    uint32_t c    = ctrl_cur[idx];
    uint32_t off8 = (g_lo >> 12) & 0xFFu;
    if (c & CT_SLOW)
        c = bus_slow_ctrl(c, off8, selected, is_pbi);

    cpu_ll_write_dedic_gpio_mask(DEDIC_OUT_CTRL_MASK, c);

    if (g_lo & (1UL << PIN_RW))
    {
        if (c & CT_DRIVE)
        {
            const uint8_t *p = bus_rd_base[idx];
```

```
            if (c & CT_RAMBO)
                p += rambo_off;
            bus_drive(p[off8]);          /* GPIO.out + GPIO.enable_w1ts */
        }
        while (cpu_ll_read_dedic_gpio_in() & DEDIC_IN_PHI2);
        if (c & CT_DRIVE)
            bus_release();
        cpu_ll_write_dedic_gpio_mask(DEDIC_OUT_CTRL_MASK, DEDIC_OUT_CTRL_IDLE);
        continue;
    }

    uint32_t last = g_lo, now;
    while ((now = GPIO.in) & (1UL << PIN_PHI2))
        last = now;
    cpu_ll_write_dedic_gpio_mask(DEDIC_OUT_CTRL_MASK, DEDIC_OUT_CTRL_IDLE);
    /* ... effetti della scrittura durante PHI1 ... */
}
```

Listing 5.2: Ciclo principale di `MonitorTask` (estratto di `main.cpp`).

5.5 Analisi del codice generato

Il listato 5.3 riporta il codice Xtensa del percorso di lettura di una pagina servita dall'ESP32 (ROM PBI), estratto dal disassemblato di `firmware.elf`. Il compilatore traduce fedelmente il sorgente: ogni accesso `volatile` ai registri GPIO è preceduto da `memw`, che attende il completamento degli accessi precedenti, per cui le quattro operazioni sulle periferiche (due letture e due scritture) si susseguono senza sovrapporsi.

```
loop:  ee.get_gpio_in   a2            ; PHI2 nel bit 0
       bbci     a2, 0, loop           ; attesa del fronte di salita
       memw
       l32i.n   a2, a6, 60            ; GPIO.in    (APB)
       memw
       l32i     a8, a6, 64            ; GPIO.in1   (APB)
       ...                            ; 16 istruzioni: indice di pagina,
       l8ui     a8, a8, 0             ;   voce della tabella ctrl_cur[idx]
       ...
       ee.wr_mask_gpio_out a8, a9     ; DEV_SEL_N / EXTSEL / MPD
       bbci     a2, 2, write          ; R/W
       ...                            ; 22 istruzioni: bus_rd_base[idx],
       l8ui     a2, a11, 0            ;   p[off8], lut_drive[byte]
       ...
       memw
       s32i.n   a8, a2, 4             ; GPIO.out        (APB)
       memw
       s32i     a8, a2, 36            ; GPIO.enable_w1ts (APB)
```

Listing 5.3: Percorso di lettura nel codice generato (indirizzi `0x403753a1`–`0x40375436`).

La tabella 5.1 riassume il conteggio. Le istruzioni sono contate a un ciclo, più due cicli di penalità per ogni salto preso e uno per ogni dipendenza di caricamento non coperta; gli accessi alle periferiche sono indicati separatamente con la latenza L_{APB}, il parametro discusso nella sezione 6.1.1.

Il conteggio conferma quanto riportato nella documentazione del progetto (16 istruzioni dopo le letture per arrivare ai segnali di controllo, 26 per `$D1xx`) e aggiunge il dato che mancava: il percorso completo fino al dato sul bus richiede $20 + 27 = 47$ cicli di sola elaborazione, più quattro accessi alle periferiche, contro i 67 cicli dell'intera fase alta di ϕ_2.

Tabella 5.1: Cicli di CPU lungo i percorsi del ciclo principale (dal disassemblato).

Tratto	**Cicli**	**Accessi APB**
attesa di ϕ_2 (`ee.get_gpio_in` + `bbci`)	3 per iterazione	—
letture e decodifica fino ai segnali di controllo	20	2 letture
idem per `$D1xx`/`$D5xx` (`CT_SLOW`)	30	2 letture
lettura del byte e pilotaggio del bus	27	2 scritture
iterazione del campionamento in scrittura	6	1 lettura

Capitolo 6

Verifica temporale

6.1 Modello

Tutti i tempi sono misurati dal fronte di ϕ_2 sul connettore dell'Atari. Un percorso è la somma di ritardi hardware (tabella 3.2) e di tempi di esecuzione (tabella 5.1, ciclo di CPU $t_c = 4{,}167\,\text{ns}$). Per ogni percorso si calcola il *margine* (*slack*): il limite meno il tempo calcolato. Un margine negativo è una violazione.

Si considerano due casi:

tipico ritardi tipici, ϕ_2 simmetrico, attesa del fronte a metà iterazione;

peggiore ritardi massimi, sincronizzatore di ingresso a due cicli di APB, fronte rilevato all'ultimo ciclo dell'iterazione di attesa, fase alta di ϕ_2 ridotta di 10 ns.

6.1.1 La latenza degli accessi alle periferiche

La CPU dell'ESP32-S3 raggiunge i registri GPIO attraverso il bus delle periferiche, che funziona a 80 MHz. La latenza di una lettura di `GPIO.in` non è specificata nei documenti di Espressif disponibili: la documentazione del progetto indica circa 6 cicli di CPU. Nel modello la latenza L_{APB}, uguale per letture e scritture, è un parametro con tre valori di riferimento:

Scenario	L_{APB} **(cicli)**	L_{APB} **(ns)**
ottimistico (valore del progetto)	6	25
nominale	12	50
pessimistico	20	83

La sezione 9.1 descrive come misurarla con un programma di prova sulla sola scheda ESP32, senza Atari.

6.2 Rilevamento del fronte di ϕ_2

Il fronte attraversa U6 ($\leq 6{,}5$ ns) e il sincronizzatore del GPIO (0–25 ns); l'iterazione di attesa dura 3 cicli, per cui il fronte è visto con un'incertezza di 12,5 ns. Il rilevamento richiede quindi 27 ns nel caso tipico e 48 ns nel caso peggiore.

6.3 Ciclo di lettura servito dall'ESP32

È il caso delle letture della ROM PBI, della RAM PBI e della finestra RAMbo. Il percorso è

$$t_{\text{dato}} = t_{\text{ril}} + (20 + 27 + 4\,L_{APB})\,t_c + t_{\text{pad}} + t_{U19} \tag{6.1}$$

e va confrontato con il limite (2.1). La figura 6.1 mostra il ciclo nel caso tipico con la latenza ottimistica, la tabella 6.1 tutti gli scenari.

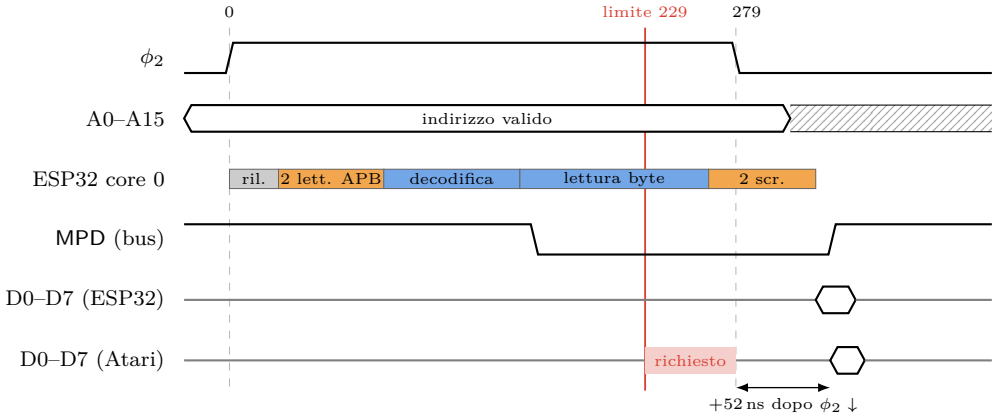


Figura 6.1: Lettura della ROM PBI servita dall'ESP32, caso tipico con $L_{APB} = 6$ cicli. Il dato arriva sul bus 331 ns dopo la salita di ϕ_2, quando il 6502 ha già campionato il bus.

Tabella 6.1: Lettura servita dall'ESP32: istante del dato sul bus e margine.

	Caso tipico			Caso peggiore		
Scenario	t_{dato}	**NTSC**	**PAL**	t_{dato}	**NTSC**	**PAL**
ottimistico	331	−101	−99	356	−137	−135
nominale	431	−201	−199	456	−237	−235
pessimistico	564	−335	−332	590	−370	−368

Tempi e margini in nanosecondi; limite 229/232 ns (tipico) e 219/222 ns (peggiore).

Risultato. Il percorso viola il setup del 6502 in tutti gli scenari, di circa 100 ns nel più favorevole. Il risultato non dipende dall'incertezza su L_{APB}: anche con accessi alle periferiche istantanei i 47 cicli di elaborazione (196 ns) più rilevamento e ritardi di uscita portano il dato a circa 230 ns, oltre il limite di 229 ns. Le conseguenze sono:

- la lettura dell'intestazione della ROM PBI (`$D803`, `$D80B`) restituisce il valore del bus non pilotato (`$FF` sugli XL, che hanno resistenze di pull-up sul bus dati [1, §3.3]): il sistema operativo non riconosce il dispositivo;
- le letture della finestra RAMbo e della RAM PBI restituiscono dati errati;
- il dato pilotato dopo la discesa di ϕ_2 resta sul bus fino a quando l'ESP32 rileva il fronte e lo rilascia, cioè fino a circa 60 ns nella fase ϕ_1 successiva.

Il passaggio da NTSC a PAL guadagna soltanto 2,6 ns.

6.4 Ciclo di lettura servito dalla VERA

Per i registri `$D100`–`$D11F` l'ESP32 deve solo abbassare DEV_SEL_N: la VERA ha già il dato pronto (registri e prefetch) e lo porta sul bus attraverso U19. Il percorso passa per `CT_SLOW` (30 cicli e due letture) e prosegue nella FPGA:

$$t_{\mathrm{VERA}} = t_{\mathrm{ril}} + (30 + 2\,L_{APB})\,t_c + t_{\mathrm{pad}} + t_{\mathrm{FPGA}} + t_{U19}. \tag{6.2}$$

Tabella 6.2: Lettura di un registro VERA: istante del dato sul bus e margine.

	Caso tipico			**Caso peggiore**		
Scenario	t_{VERA}	**NTSC**	**PAL**	t_{VERA}	**NTSC**	**PAL**
ottimistico	220	9	12	251	−31	−29
nominale	270	−41	−38	301	−81	−79
pessimistico	337	−107	−105	367	−148	−145

Tempi e margini in nanosecondi.

Risultato. La lettura dei registri VERA è al limite: rispetta il setup solo nel caso tipico con latenza ottimistica, con un margine di 9 ns in NTSC. È il punto in cui la misura di L_{APB} è decisiva.

6.5 Cicli di scrittura

6.5.1 Campionamento del dato

In scrittura il firmware rilegge `GPIO.in` finché ϕ_2 è alto e usa l'ultimo campione. Il campione contiene nello stesso registro ϕ_2 e i dati, quindi è coerente. Il periodo di campionamento è $(L_{APB} + 6)\,t_c$: da 50 ns a 108 ns. L'ultimo campione con ϕ_2 alto è preso al più un periodo (più il sincronizzatore) prima della discesa e deve seguire l'istante in cui il dato del 6502 è valido ($t_{MDS} + t_{U19}$). Il margine, nel caso peggiore, è di 88 ns (ottimistico), 63 ns (nominale) e 30 ns (pessimistico) in NTSC: **il campionamento delle scritture è corretto** in tutti gli scenari.

6.5.2 Effetti della scrittura durante ϕ_1

Dopo la discesa di ϕ_2 il firmware applica gli effetti: memorizza il dato nella RAM PBI o nel banco RAMbo (circa 12 cicli), aggiorna il latch o le copie di PORTB e PBCTL. Il percorso comune rientra ampiamente nei 279 ns di ϕ_1. Due percorsi meno frequenti sono più lunghi:

- ogni scrittura nelle pagine `$D1xx`, `$D3xx` e `$D5xx`, quindi anche *ogni scrittura in un registro della VERA*, chiama `bus_idx_to_page()`. La funzione è dichiarata `static inline` ma il compilatore non l'ha espansa: è una chiamata (`callx8`) a codice in *flash* (`0x42023830`), eseguito attraverso la cache delle istruzioni condivisa con il core 1. Un *cache miss* costa il caricamento di una linea dalla flash SPI, nell'ordine del microsecondo, cioè due o più cicli di bus persi;
- il cambio di latch registra un evento (`log_send()` con `xTaskGetTickCount()`): un centinaio di cicli, che possono far perdere il fronte di salita del ciclo seguente.

Perdere il fronte non è di per sé un errore: il ciclo seguente viene trattato in ritardo. È un errore se quel ciclo è una lettura servita dall'ESP32 o una scrittura a un registro VERA, come avviene nei cicli di scrittura consecutivi su `DATA0`.

6.5.3 Hold del dato verso la VERA

La VERA cattura il dato di scrittura sul fronte di discesa di `bus_write = !cs_n && !wr_n`, cioè quando $\overline{\text{mWE}}$ risale dopo la discesa di ϕ_2 [14]. Lo stesso fronte di ϕ_2 disabilita U19 attraverso U23, e da quel momento il dato al pin della FPGA non è più pilotato. La figura 6.2 mostra la corsa:

$$t_{\overline{\text{WE}}\uparrow} = t_{U6} + t_{\text{HC00}}, \qquad t_{\text{fine dato}} = \min\left(t_{HW} + t_{U19},\ t_{\text{HCT04}} + t_{\text{dis},U19}\right). \tag{6.3}$$

Il margine di hold è -2 ns con i ritardi tipici e -32 ns nel caso peggiore (porte veloci per U23 e U19, lente per U6 e U5): **la scrittura nei registri VERA dipende dalla carica residua sulle linee**, non da un margine garantito. Il problema è indipendente dal firmware ed era già stato segnalato nella documentazione dell'hardware [14, §2.2].

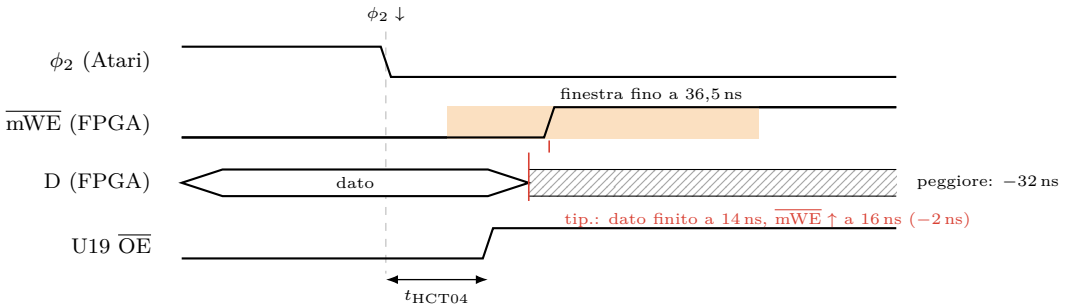


Figura 6.2: Hold del dato di scrittura alla FPGA (scala: 1 ns = 1,6 mm). Il dato smette di essere pilotato quando U19 si disabilita, mentre la VERA lo cattura alla salita di $\overline{\text{mWE}}$, che può arrivare fino a 36,5 ns dopo la discesa di ϕ_2.

6.6 Asserzione di EXTSEL e MPD

Il firmware asserisce EXTSEL e MPD dopo aver letto l'indirizzo, cioè dopo la salita di ϕ_2: al bus arrivano a 168 ns nel caso tipico ottimistico e fino a 304 ns nel caso peggiore pessimistico. Un decodificatore combinatorio li asserirebbe circa 120 ns *prima* della salita di ϕ_2. Nella fase alta di ϕ_2 la logica dell'Atari ha già abilitato la RAM dinamica o la ROM del Math Pack:

- in lettura, la RAM o la ROM interna pilota il bus finché EXTSEL/MPD non la disabilita: per alcune decine di nanosecondi due dispositivi pilotano il bus dati (contesa);
- in scrittura nella finestra RAMbo, se CAS della RAM interna è già sceso quando arriva EXTSEL, la scrittura raggiunge anche la RAM di base a `$4000`–`$7FFF`, che dovrebbe restare nascosta: il contenuto della memoria principale viene alterato da scritture destinate ai banchi estesi.

L'esatta posizione di RAS/CAS rispetto a ϕ_2 dipende dal modello (800XL con logica discreta, 130XE con FREDDIE) e va misurata. In ogni caso un segnale di sovrapposizione asserito oltre 150 ns dopo la salita di ϕ_2 è fuori dalla prassi dei dispositivi PBI esistenti e va considerato non conforme finché la misura non dimostra il contrario.

6.7 Interruzioni sul core 0

Il core 0 esegue solo il ciclo principale, ma non ha le interruzioni disabilitate: il tick di FreeRTOS (`CONFIG_FREERTOS_HZ = 1000`) interrompe il ciclo una volta al millisecondo. Ingresso e uscita dall'interruzione (salvataggio della finestra di registri), gestore del tick ed eventuale riprogrammazione richiedono alcune centinaia di cicli: con una stima di 1,5–5 μs si perdono da 3 a 9 cicli di bus ogni millisecondo (figura 6.3).

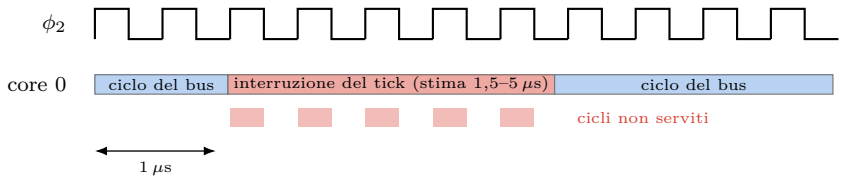


Figura 6.3: Effetto dell'interruzione del tick di FreeRTOS sul core 0.

Sono circa lo 0,2–0,5 % dei cicli. Per un programma che esegue codice dalla ROM PBI o dalla finestra RAMbo un ciclo perso è un codice operativo errato, quindi un blocco entro pochi millisecondi; per un ciclo di scrittura a un registro VERA è una scrittura persa. Disabilitare le interruzioni sul core 0 non è possibile così com'è: in ESP-IDF 4.4 il contatore dei tick è aggiornato dal core 0 e il watchdog delle interruzioni controlla entrambi i core (`CONFIG_ESP_INT_WDT_CHECK_CPU1 = 1`). La soluzione è discussa nella sezione 8.3.

6.8 Sintesi grafica

Le figure 6.4 e 6.5 riportano il margine dei percorsi in funzione di L_{APB}: la parte sotto lo zero è la zona di violazione.

Tabella 6.3: Esito della verifica temporale.

Percorso	Esito	Nota
Lettura ROM/RAM PBI e RAMbo	non conforme	violazione di circa 100–370 ns in ogni scenario, PAL e NTSC
Lettura registri VERA	al limite	conforme solo con $L_{APB} \approx 6$ cicli e ritardi tipici
Campionamento scritture	conforme	margine ≥ 30 ns anche nel caso peggiore
Effetti di scrittura in ϕ_1	a rischio	chiamata a codice in flash per `$D1xx`/`$D3xx`/`$D5xx`
Hold scrittura VERA	non conforme	-32 ns (hardware)
EXTSEL/MPD	da misurare	asseriti dopo $\phi_2 \uparrow$, contesa probabile
Interruzioni sul core 0	non conforme	3–9 cicli persi ogni ms

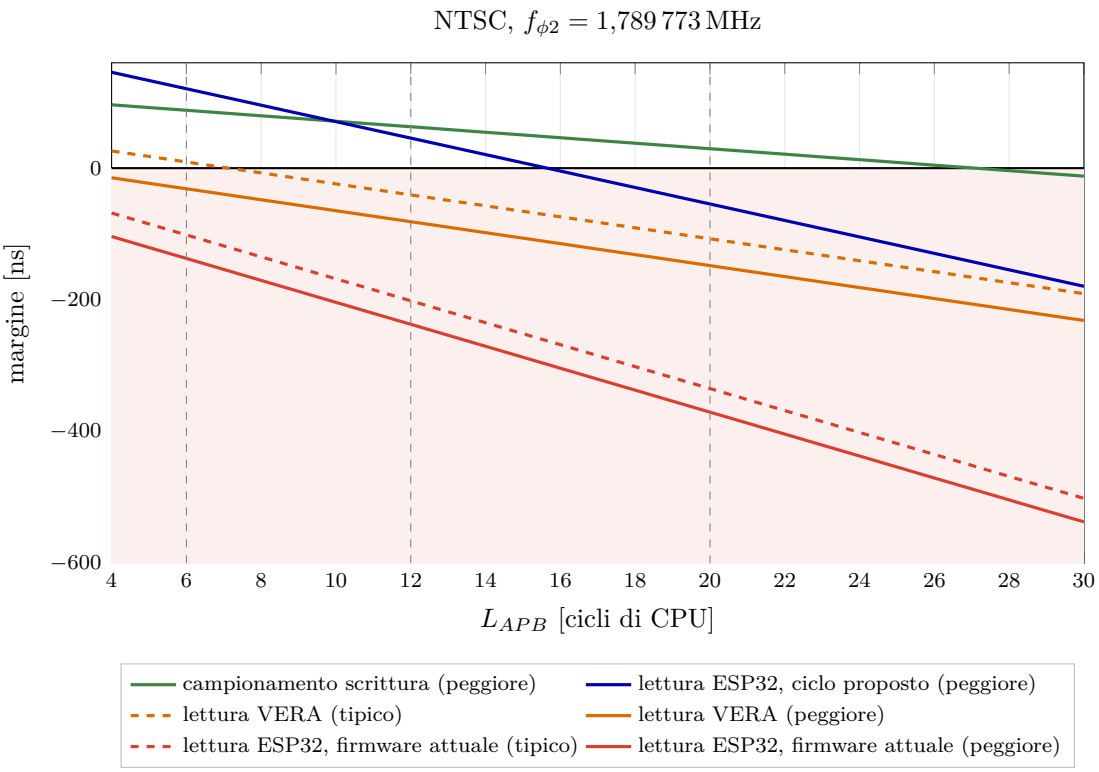


Figura 6.4: Margine dei percorsi critici in funzione della latenza APB (NTSC).

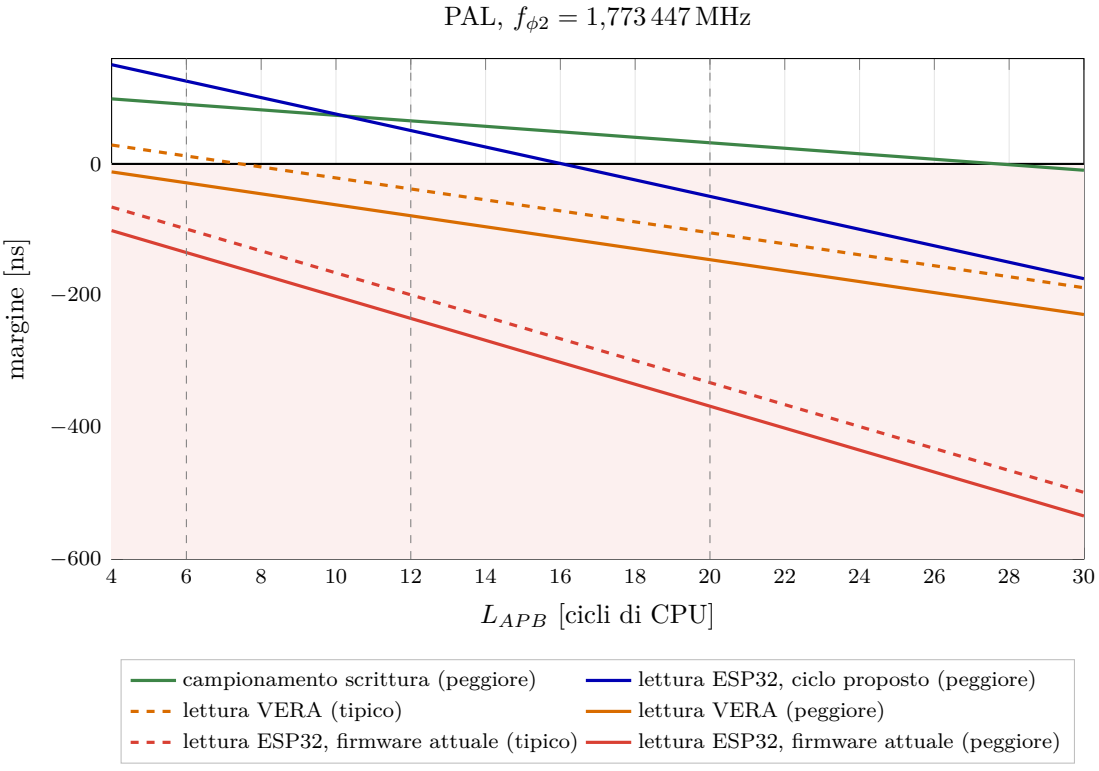


Figura 6.5: Margine dei percorsi critici in funzione della latenza APB (PAL).

Capitolo 7

Difetti non temporali

L'analisi del codice generato ha messo in luce tre difetti che non dipendono dalle tempistiche e che impedirebbero il funzionamento anche con un ciclo principale abbastanza veloce.

7.1 Accessi a byte all'area IRAM

La ROM PBI (`pbi_driver[]`, dichiarata `IRAM_ATTR`) e l'array della RAMbo (`extended_rambo_256k[]`, nella sezione `.iram0.bss`) sono collocati agli indirizzi del bus istruzioni: `0x403793b6` e `0x40383284`. Il ciclo principale li legge e li scrive con le istruzioni `l8ui` e `s8i`. ESP-IDF considera accessibile a byte solo l'intervallo del bus dati `SOC_BYTE_ACCESSIBLE_LOW`–`HIGH` = `0x3FC88000`–`0x3FD00000` (`soc.h`, funzione `esp_ptr_byte_accessible()`): un accesso a 8 bit all'indirizzo IRAM genera un'eccezione *LoadStoreError* e il riavvio del chip. Il primo accesso del sistema operativo alla ROM PBI, la lettura di `$D803` durante la scansione dei dispositivi, provocherebbe quindi il blocco del firmware.

7.2 Sovrapposizione fra RAMbo e memoria dati

Sull'ESP32-S3 la stessa SRAM è visibile sia dal bus istruzioni (`0x40378000`–`0x403DFFFF`) sia dal bus dati (`0x3FC88000`–`0x3FCEFFFF`), con uno scostamento costante di `0x6F0000`. Il linker riserva sul bus dati lo spazio occupato dal codice in IRAM con la sezione `.dram0.dummy`, la cui dimensione è calcolata fino al simbolo `_iram_end`. La sezione `.iram0.bss` creata dall'attributo nel sorgente non è fra quelle previste dallo script di linker di ESP-IDF: viene collocata dopo `_iram_end` e non è contata. Dall'intestazione del file ELF:

Sezione	**Indirizzo**	**Dimensione**
`.dram0.dummy`	`0x3FC88000`	`0xB284` (fino a `_iram_end` = `0x40383284`)
`.dram0.data`	`0x3FC93290`	`0x45B8`
`.dram0.bss`	`0x3FC97848`	`0x14A8`
heap	da `0x3FC98CF0`	
`.iram0.bss`	`0x40383284`	`0x40000` (alias dati `0x3FC93284`–`0x3FCD3284`)

I 256 KB della RAMbo occupano quindi la stessa memoria fisica di `.data`, `.bss` e di buona parte dello heap. Con RAMBO_EN alto la chiamata `memset(extended_rambo_256k, 0, ...)` in `setup()` azzera le variabili globali, le strutture di FreeRTOS e lo stack del task in esecuzione; anche senza `memset`, ogni scrittura nella finestra RAMbo corromperebbe lo stato del firmware. Il linker non segnala nulla perché le due viste sono regioni di memoria distinte.

7.3 ROM di autotest e finestra RAMbo

Quando il bit 0 di PORTB è a 1 (ROM del sistema abilitata) e il bit 7 è a 0, la ROM di autotest appare a `$5000`–`$57FF` con priorità sulla memoria estesa [1, §3.4–3.5]. Le tabelle del firmware considerano solo il bit 4: con PORTB = `$01` (o qualunque valore con bit 0 = 1, bit 4 = 0, bit 7 = 0) l'ESP32 pilota il bus a `$5000`–`$57FF` insieme alla ROM interna, e EXTSEL non può evitarlo perché disabilita solo la RAM.

7.4 Altri punti minori

- La copia di PORTB non tiene conto del DDR: un bit configurato come ingresso appare a 1 sui pin della PIA (pull-up), mentre il firmware usa il valore scritto nel registro di uscita. Il sistema operativo configura tutti i bit come uscite, per cui il caso riguarda solo programmi che modificano il DDR.
- Il reset dell'Atari riporta la PIA allo stato iniziale (tutti ingressi, quindi PORTB letto come `$FF`), ma il firmware conserva l'ultimo valore osservato fino alla nuova inizializzazione da parte del sistema operativo.
- Il commento all'inizio di `main.cpp` descrive ancora i traslatori TXS0108E della revisione precedente; la scheda usa 74LVC4245.

Capitolo 8

Proposte di correzione

8.1 Correzioni della collocazione in memoria

Le due correzioni dei difetti delle sezioni 7.1 e 7.2 richiedono due righe:

```
/* include/pbi-driver.h */
const uint8_t DRAM_ATTR pbi_driver[] = {            /* era IRAM_ATTR */

/* src/main.cpp: .bss ordinaria nella DRAM, accessibile a byte */
static uint8_t extended_rambo_256k[256 * 1024];     /* senza section(".iram0.bss") */
```

Listing 8.1: Collocazione corretta di ROM PBI e RAMbo.

La variante è stata compilata: il firmware occupa 289 376 dei 327 680 byte della regione dati (88,3 %), la RAMbo si trova a `0x3FC9784D`, la ROM a `0x3FC94D5C` e lo heap parte da `0x3FCD8CF0`. La memoria residua per lo heap è limitata e va verificata all'avvio (`heap_caps_get_free_size()`). Nello stesso intervento conviene rendere `bus_idx_to_page()` `__attribute__((always_inline))`, o meglio confrontare direttamente l'indice di pagina con le costanti precalcolate per `$D1`, `$D3` e `$D5`, eliminando la chiamata a codice in flash.

8.2 Ciclo principale con decodifica durante ϕ_1

Il difetto temporale di fondo è che il firmware inizia a lavorare alla salita di ϕ_2, mentre l'indirizzo è stabile da circa 140 ns. La proposta riorganizza il ciclo in modo che tutto ciò che dipende dall'indirizzo avvenga in ϕ_1:

1. alla discesa di ϕ_2 (fine del ciclo precedente) si applicano gli effetti della scrittura precedente;

2. si attende che l'indirizzo sia garantito stabile: t_{ADS} più ritardo di U3/U4 e sincronizzatore, misurati dal fronte di discesa con il contatore di cicli `CCOUNT`;

3. si campiona l'indirizzo con *una sola* lettura APB: ϕ_2, $R/\overline{W}$ e A10–A15 passano al modulo Dedicated GPIO, che ha otto canali di ingresso e si legge in un ciclo; A0–A9 restano in `GPIO.in`. Non serve modificare la scheda, perché i canali Dedicated GPIO si collegano a qualunque pin attraverso la matrice GPIO;

4. si decodifica e si asseriscono EXTSEL/MPD/DEV_SEL_N prima della salita di ϕ_2, come farebbe un decodificatore combinatorio;

5. nei cicli di lettura serviti dall'ESP32 si legge il byte da una tabella di puntatori per pagina (che include già l'offset del banco RAMbo) e si pilotano i pin dati. Il bus Atari resta isolato da U19 fino alla salita di ϕ_2, per cui pilotare i pin in anticipo è sicuro;

6. alla salita di ϕ_2 resta solo l'attesa della discesa (lettura) o il campionamento del dato (scrittura).

La figura 8.1 mostra il ciclo risultante nel caso peggiore con latenza nominale e la tabella 8.1 i margini.

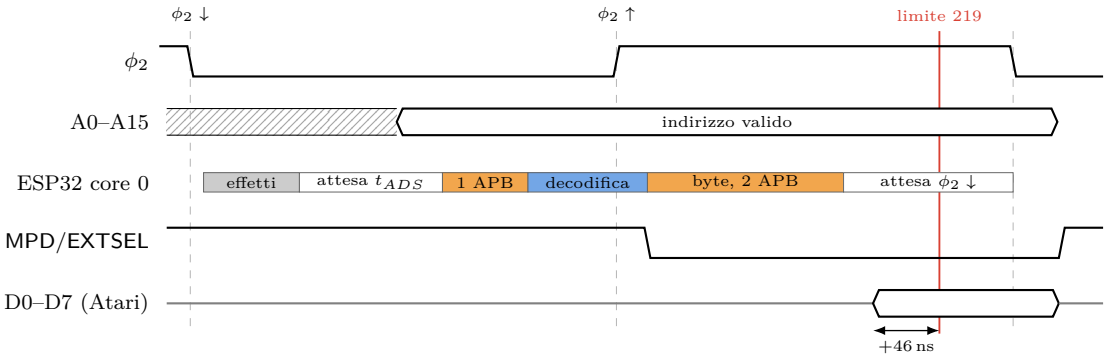


Figura 8.1: Ciclo di lettura con decodifica durante ϕ_1, caso peggiore in NTSC con $L_{APB} = 12$ cicli: i segnali di controllo arrivano 21 ns dopo la salita di ϕ_2, il dato a 174 ns.

Tabella 8.1: Ciclo proposto, caso peggiore: istanti rispetto a $\phi_2 \uparrow$ e margine del dato.

	NTSC			PAL		
Scenario	controlli	dato	margine	controlli	dato	margine
ottimistico	−4	99	121	−6	96	126
nominale	21	174	46	19	171	51
pessimistico	55	274	−54	52	271	−49

Valori in nanosecondi.

Il ciclo proposto rientra nei limiti per latenze fino a circa 15 cicli; nello scenario pessimistico resta in violazione, perché il campionamento attende il valore massimo di t_{ADS} e lascia poco più di 100 ns prima della salita di ϕ_2. Anche i segnali di sovrapposizione arrivano in prossimità della salita di ϕ_2 e non 120 ns prima come con un decodificatore combinatorio: il miglioramento è grande, ma la conformità dipende ancora dalla misura di L_{APB} e di t_{ADS} reale.

8.3 Isolamento del ciclo principale dalle interruzioni

Per eliminare la perdita di cicli della sezione 6.7 il ciclo principale deve girare a interruzioni disabilitate. In ESP-IDF 4.4 questo richiede:

- spostare `MonitorTask` sul core 1 e il log sul core 0, che deve continuare ad aggiornare il contatore dei tick;

- disabilitare nel task le interruzioni fino al livello del watchdog (`XTOS_SET_INTLEVEL`) e alimentare periodicamente il watchdog delle interruzioni, oppure compilare il framework con `CONFIG_ESP_INT_WDT_CHECK_CPU1 = n` (Arduino come componente di ESP-IDF);

- evitare nel ciclo qualunque chiamata a funzioni di FreeRTOS: il timestamp degli eventi può essere il registro `CCOUNT`.

8.4 Correzioni minori del firmware

- Aggiungere alle tabelle lo stato «autotest visibile» (`(portb & 0x81) == 0x01`) ed escludere le pagine `$50`–`$57` dalla finestra RAMbo in quello stato (otto tabelle invece di quattro, 2 KB).
- Calcolare il PORTB effettivo come `out | ~ddr`, osservando anche le scritture al DDR.

8.5 Correzioni hardware

Restano necessarie le modifiche già individuate nella documentazione della scheda [14], che l'analisi conferma:

- abilitare U19 solo nei cicli in cui la scheda risponde o in scrittura, invece che in ogni ciclo con ϕ_2 alto: oggi U19 pilota il bus dati dell'Atari con livelli indefiniti in ogni lettura di RAM o ROM interna;
- rendere positivo l'hold della scrittura verso la VERA (sezione 6.5.3), ritardando la disabilitazione di U19 rispetto alla salita di $\overline{\text{mWE}}$ e sostituendo il 74HC00 con una porta più veloce a 3,3 V (74LVC00);
- collegare l'uscita IRQ della VERA, che è push-pull, attraverso uno stadio open-drain (74LVC1G07).

8.6 Alternativa architetturale: ROM, RAM e RAMbo nella FPGA di interfaccia

Il repository contiene anche un'implementazione dello stesso controllo del bus in VHDL per una FPGA iCE40UP5K di interfaccia (`C6502_Interface.vhd`), con ROM e RAM PBI in BRAM e la RAMbo in una SRAM asincrona esterna da 55 ns. In logica combinatoria la decodifica segue l'indirizzo di poche decine di nanosecondi, EXTSEL e MPD sono validi durante ϕ_1 e il dato della SRAM arriva sul bus circa 30–40 ns dopo la salita di ϕ_2, con un margine di quasi 200 ns. Se le misure della sezione 9.1 confermassero latenze APB elevate, questa sarebbe la strada più solida: all'ESP32 resterebbero l'avvio, il reset e la diagnostica.

Capitolo 9

Piano di verifica sperimentale

9.1 Misura della latenza APB

La latenza di `GPIO.in` si misura su una scheda ESP32-S3 qualsiasi, senza Atari, con il contatore di cicli della CPU. Il listato 9.1 esegue 1000 letture e riporta i cicli medi per lettura; la stessa struttura con `GPIO.out` misura le scritture.

```
#include <Arduino.h>
#include <soc/gpio_struct.h>
#include <xtensa/hal.h>

static void IRAM_ATTR bench(void)
{
    uint32_t sink = 0;
    portDISABLE_INTERRUPTS();
    uint32_t t0 = xthal_get_ccount();
    for (int i = 0; i < 1000; i++)
        sink += GPIO.in;                /* memw + l32i, come nel firmware */
    uint32_t t1 = xthal_get_ccount();
    portENABLE_INTERRUPTS();
    Serial.printf("GPIO.in: %.2f cicli (sink %u)\n", (t1 - t0) / 1000.0f, sink);
}

void setup() { Serial.begin(115200); delay(500); bench(); }
void loop()  {}
```

Listing 9.1: Misura della latenza di lettura di `GPIO.in`.

Il valore misurato, meno i 3–4 cicli di overhead dell'iterazione, sostituisce `SCENARIOS` in `timing_model.py`; ricompilando la tesi si aggiornano tabelle e grafici. Lo stesso programma, con il core 1 impegnato a stampare sulla seriale, mostra l'effetto della contesa sul bus delle periferiche.

9.2 Misure sul bus con analizzatore logico

Con l'analizzatore logico (almeno 200 MS/s per una risoluzione di 5 ns) e la scheda collegata all'Atari:

1. duty cycle di ϕ_2 sul connettore, in PAL e NTSC;
2. t_{ADS} reale: ritardo fra la discesa di ϕ_2 e la stabilità di A0–A15;
3. ritardo fra la salita di ϕ_2 e DEV_SEL_N, EXTSEL, MPD;

4. ritardo fra la salita di ϕ_2 e il dato sul connettore in una lettura di `$D803`, confrontato con la discesa di ϕ_2;

5. posizione di RAS/CAS della RAM interna rispetto a ϕ_2 (sonda sulla scheda madre) per stabilire il limite reale di EXTSEL;

6. hold del dato al pin della FPGA rispetto alla salita di $\overline{\text{mWE}}$;

7. cicli non serviti: un GPIO libero (47 o 48) commutato a ogni iterazione del ciclo principale rende visibili le pause del tick e dei cache miss.

9.3 Prove funzionali

Dopo le misure, i programmi di prova del progetto [14] verificano il comportamento complessivo: riconoscimento della ROM PBI all'avvio, rilevamento della VERA (`vera_detect()`), `TESTFX.COM` (atteso 36 PASS), prove della RAMbo con un programma che scrive e rilegge tutti i 16 banchi controllando che la memoria di base a `$4000`–`$7FFF` resti invariata, e `TESTIRQ.COM` dopo il montaggio dello stadio open-drain.

Capitolo 10

Conclusioni

Il modulo Vera-Module-RBL-XE affida a un ESP32-S3 l'intera decodifica del bus di un Atari XL/XE: registri della VERA, dispositivo PBI ed emulazione della RAMbo 320K. La logica è corretta: il latch di selezione usa solo il proprio bit come richiede il protocollo PBI, la scheda non risponde quando non è selezionata, il bank switching segue la configurazione «320K Rambo» (bit 2, 3, 5, 6 per il banco, bit 4 per l'abilitazione di CPU e ANTIC) e l'osservazione di PBCTL distingue correttamente le scritture al DDR da quelle a PORTB. Anche il campionamento dei cicli di scrittura rispetta le tempistiche del 6502.

La verifica temporale mostra però che, a 1,77 e a 1,79 MHz, il firmware attuale non è conforme per i cicli di lettura serviti dall'ESP32: l'elaborazione inizia alla salita di ϕ_2 e i 47 cicli di istruzioni più quattro accessi alle periferiche non possono rientrare nei 279 ns della fase alta meno il setup del processore. La differenza fra PAL e NTSC (2,6 ns) è irrilevante rispetto al deficit. Le letture dei registri VERA sono al limite e dipendono dalla latenza APB reale; le interruzioni sul core 0 fanno perdere alcuni cicli ogni millisecondo. L'analisi del codice generato ha inoltre rivelato che ROM PBI e RAMbo sono in un'area non accessibile a byte e che la RAMbo si sovrappone alla memoria dati del firmware: difetti che bloccherebbero la scheda alla prima lettura della ROM o all'avvio con la RAMbo abilitata.

Le correzioni della collocazione in memoria sono immediate e già verificate in compilazione. La riorganizzazione del ciclo principale con decodifica durante ϕ_1, una sola lettura APB e pilotaggio anticipato del bus porta il margine a circa +46 ns con latenza nominale, ma resta dipendente dalle misure. Se le misure di L_{APB} confermassero valori alti, la soluzione robusta è spostare ROM, RAM e RAMbo nella FPGA di interfaccia già progettata, lasciando all'ESP32 i compiti non critici.

Bibliografia

[1] A. Lee, *Altirra Hardware Reference Manual*, versione 07/05/15, 2015.

[2] Rockwell International, *R6500 Microprocessors (CPU) – R6502, R6502A*, data sheet, caratteristiche AC dei cicli di lettura e scrittura.

[3] MOS Technology, *MCS6500 Microcomputer Family Hardware Manual*, 1976.

[4] Espressif Systems, *ESP32-S3 Series Datasheet*, v2.2.

[5] Espressif Systems, *ESP32-S3 Technical Reference Manual*, capitoli «System and Memory», «IO MUX and GPIO Matrix» e «Dedicated GPIO».

[6] Espressif Systems, *ESP-IDF Programming Guide v4.4*: «Dedicated GPIO» e «Memory Types»; `soc/esp32s3/include/soc/soc.h`.

[7] NXP Semiconductors, *74LVC4245A – Octal dual supply translating transceiver; 3-state*, data sheet.

[8] Texas Instruments, *SN74HCT04 Hex Inverters*, data sheet.

[9] Texas Instruments, *SN74HC00 Quadruple 2-Input Positive-NAND Gates*, data sheet.

[10] Lattice Semiconductor, *iCE40 UltraPlus Family Data Sheet*.

[11] F. van den Hoef, *VERA – Versatile Embedded Retro Adapter*, HDL 48.0.1, progetto Commander X16.

[12] ICD Inc., *RAMBO XL Memory Upgrade – manuale*.

[13] Progetto Vera-Module-RBL-XE, `PIN-MAPPING.md` e `firmware/MCU/README.md`.

[14] Progetto Vera-Module-RBL-XE, `VERA-ATARI-HW-REQUIREMENTS.md`, 2026-10-06.

Appendice A

Il modello temporale

Lo script `timing_model.py` contiene tutti i parametri delle tabelle 2.2, 3.2 e 5.1 e calcola i percorsi descritti nel capitolo 6. Produce `data/numbers.tex`, con una macro LaTeX per ogni valore citato nel testo, e i file `data/slack_ntsc.dat` e `data/slack_pal.dat` per i grafici. Per aggiornare la tesi dopo una misura:

```
python3 timing_model.py      # rigenera data/
latexmk -pdf tesi.tex        # oppure: pdflatex tesi.tex (due volte)
```